**2. Data Analytics I**

Create a Linear Regression Model using Python/R to predict home prices using "coffee_shop_revenue" Dataset.

1. Print all predictor variables and dependent variables in the data set.
2. Understand the relationship between each predictor variable and dependent variable. Draw the plot.
3. Build simple linear regression model for predicting Daily Revenue based on various influencing factors such as Number of Customers Per Day, Average Order Value, Operating Hours Per Day, Number of Employees, Marketing Spend Per Day, Location Foot Traffic (people/hour). Keep 80% samples for training and remaining for testing.
4. Show the accuracy on the test set.
5. Build multiple linear regression model. Compare MSE of simple linear and multiple linear regression.

Implement linear regression algorithm with gradient descent optimization.

•The program should be generic, should work for any dataset

•Print all the predictor variables and dependent variables in the given

dataset

•Understand the relationship between each predictor variable and the

dependent variable; draw the plot.

•Keep 80% of samples for training and rest for testing

•Print the regression parameters after training.

•Show the accuracy on the test set

```
# Import necessary libraries

import pandas as pd

import numpy as np
```

```
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# Step 1: Load the coffee_shop_revenue dataset
df = pd.read_csv("coffee_shop_revenue.csv")

# Step 2: Print all predictor variables and dependent variable
print("Predictor Variables and Dependent Variable:")
print(df.columns)

# Assume the last column is the dependent variable "Daily_Revenue"
predictors = df.drop(columns=['Daily_Revenue'])
dependent = df['Daily_Revenue']

# Step 3: Understand the relationship between each predictor variable and dependent variable
# Plot the relationships using pairplot or scatter plots
sns.pairplot(df)
plt.show()

# Step 4: Simple Linear Regression Model - Let's start with one predictor
# Split the data into training and testing sets (80% training, 20% testing)
X = predictors[['Number_of_Customers_Per_Day']]  # You can replace this with any other column for simple linear regression
y = dependent
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train a simple linear regression model
simple_model = LinearRegression()
```

```
simple_model.fit(X_train, y_train)

# Make predictions and evaluate the model
y_pred_simple = simple_model.predict(X_test)
mse_simple = mean_squared_error(y_test, y_pred_simple)
print(f"Mean Squared Error for Simple Linear Regression: {mse_simple}")

# Step 5: Multiple Linear Regression Model
X_multiple = predictors  # Use all the predictor variables for multiple linear regression
X_train, X_test, y_train, y_test = train_test_split(X_multiple, y, test_size=0.2, random_state=42)

# Train a multiple linear regression model
multiple_model = LinearRegression()
multiple_model.fit(X_train, y_train)

# Make predictions and evaluate the model
y_pred_multiple = multiple_model.predict(X_test)
mse_multiple = mean_squared_error(y_test, y_pred_multiple)
print(f"Mean Squared Error for Multiple Linear Regression: {mse_multiple}")

# Step 6: Compare MSE of Simple and Multiple Linear Regression
print("\nComparison of MSE:")
print(f"Simple Linear Regression MSE: {mse_simple}")
print(f"Multiple Linear Regression MSE: {mse_multiple}")

# Step 7: Implementing Linear Regression Algorithm with Gradient Descent Optimization
def gradient_descent(X, y, learning_rate=0.01, iterations=1000):
    m, n = X.shape
    X_b = np.c_[np.ones((m, 1)), X]  # Add bias term
    theta = np.random.randn(n + 1, 1)  # Initialize theta
```

```
    for _ in range(iterations):
        gradients = 2/m * X_b.T.dot(X_b.dot(theta) - y)
        theta -= learning_rate * gradients

    return theta

# Prepare the data (add intercept term)
X_b = np.c_[np.ones((X_train.shape[0], 1)), X_train]
y_train_reshaped = y_train.values.reshape(-1, 1)

# Train the model using gradient descent
theta = gradient_descent(X_b, y_train_reshaped, learning_rate=0.01, iterations=1000)
print("\nTheta values from Gradient Descent:")
print(theta)

# Make predictions using gradient descent model
X_test_b = np.c_[np.ones((X_test.shape[0], 1)), X_test]
y_pred_gd = X_test_b.dot(theta)

# Evaluate the model
mse_gd = mean_squared_error(y_test, y_pred_gd)
print(f"Mean Squared Error for Gradient Descent Model: {mse_gd}")

# Step 8: Visualizing the prediction results for multiple linear regression
plt.figure(figsize=(10, 6))
plt.scatter(y_test, y_pred_multiple)
plt.xlabel('Actual Daily Revenue')
plt.ylabel('Predicted Daily Revenue')
plt.title('Multiple Linear Regression: Actual vs Predicted')
plt.show()
```

```
# You can visualize and evaluate the models further as needed.
```